Git e GitHub

DOCENTE: LETÍCIA SHIMIZU
GITHUB: LETICIASHIMIZU

AGENDA DE HOJE

Aula: Introdução ao GitHub

Objetivo da Aula

- Entender o que é Git e GitHub
- Compreender conceitos fundamentais (repositório, commit, branch, clone etc.)
- Instalar as ferramentas necessárias
- Criar conta e publicar o primeiro projeto
- Executar um fluxo básico de versionamento

1) O que é Git?

Git é um sistema de controle de versão.

Ele serve para:

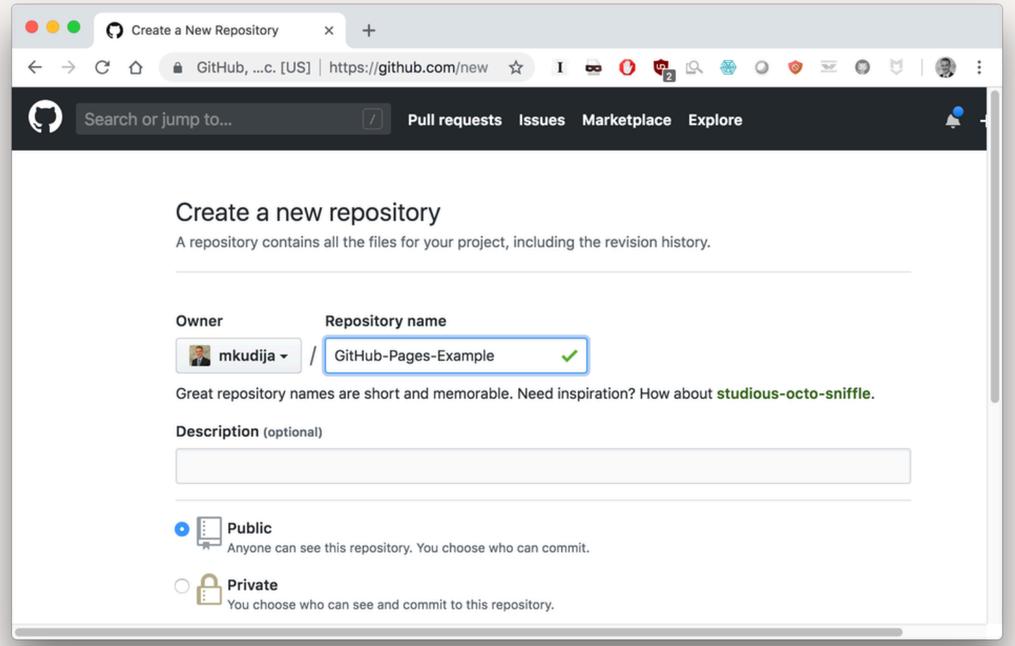
- Controlar alterações no código
- Voltar para versões anteriores
- Trabalhar em equipe
- Organizar histórico de desenvolvimento

Pense assim:

Git é como um “histórico inteligente” do seu projeto.

2) O que é GitHub?

GitHub é uma plataforma online que hospeda repositórios Git.



Ele permite:

- Armazenar projetos na nuvem
- Compartilhar código
- Trabalhar colaborativamente
- Criar portfólio profissional

Resumo:

- Git = ferramenta de controle
- GitHub = plataforma na internet que usa Git

3) Conceitos Fundamentais

Repositório (Repository)

É a pasta do projeto controlada pelo Git.

Commit

É um “registro” das alterações feitas no projeto.
Exemplo:
“Adicionado formulário de cadastro”

Branch

É uma ramificação do projeto.

- main → versão principal
- nova-feature → nova funcionalidade

Clone

Copiar um repositório da internet para o seu computador.

Push

Enviar alterações do computador para o GitHub.

Pull

Baixar atualizações do GitHub para o computador.

Downloads necessários:

Git

Site oficial: https://git-scm.com/

Baixar:

Windows → Download for Windows

Instalar tudo como padrão (Next, Next, Next)

Após instalar:

Abrir o Git Bash e testar:

Criar conta no GitHub

https://github.com

Criar conta

Confirmar e-mail

Escolher username profissional

Dica importante:

O username será visto por recrutadores. Nada de apelidos infantis.

Dica Final

GitHub hoje é currículo.
Quem quer trabalhar como:

- Desenvolvedor
- Analista
- QA
- DevOps

Precisa ter GitHub ativo.
Não é opcional.

Git e GitHub

LET´S GO!

Aula: Uso Diário do GitHub com Git Bash (Do Zero ao Fluxo Profissional)

Objetivo da aula

- Criar repositório corretamente
- Trabalhar localmente com Git Bash
- Versionar código de forma organizada
- Enviar para o GitHub
- Trabalhar com branches
- Atualizar projeto
- Resolver problemas comuns

Antes de tudo: Mentalidade correta

Saiba que:
Git não é só comando.
É organização, histórico e profissionalismo.

Entenda que:

- Cada commit conta uma história
- Nome ruim de commit = falta de profissionalismo
- Repositório mal organizado = ponto negativo em entrevista

Estrutura organizada

Antes de qualquer comando, vamos organizar as pastas:
Exemplo de estrutura:

📂 Pasta principal

```
Desktop/
└── Projetos/
```

Ou:

```
Documentos/
└── Projetos/
```

👉 Nome padrão:

```
Projetos
```

📂 2) Organização por área ou curso

Dentro de `Projetos`, você pode separar por contexto:

```
Projetos/
├── Aulas/
├── Pessoais/
├── Estudos/
├── Freelance/
```

📚 Exemplo para seus alunos (ideal)

```
Projetos/
└── Aulas/
    ├── Aula-GitHub/
    ├── Aula-CSharp/
    ├── Aula-BancoDados/
```

Estrutura de Projetos

📦 3) Estrutura de cada projeto

Cada projeto deve ter sua própria pasta:

```
Aula-GitHub/
│
├── src/
├── docs/
├── README.md
├── .gitignore
```

Significados:

- `README.md` → apresentação do projeto
- `src` → código
- `docs` → documentação
- `.gitignore` → arquivos que não devem subir

4 Padrão de nomenclatura (muito importante)

Nome de repositório Regras:

- Tudo minúsculo
- Sem espaço
- Usar hífen

correto:

Código

```
sistema-vendas
aula-git-turma-a
controle-estoque
```

errado:

Código

```
Sistema Vendas
Projeto Final 2025
teste123!!!
```

5 Nome de commits

Padrão profissional:

Formato ideal:

Código

```
tipo: descrição curta
```

Exemplos:

Código

```
feat: adiciona tela de login
fix: corrige cálculo do total
docs: atualiza README
refactor: melhora estrutura do código
```

Isso mostra maturidade técnica.

Criando a Estrutura na sua máquina!

📁 Pasta principal

```
Desktop/
└── Projetos/
```

Ou:

```
Documentos/
└── Projetos/
```

👉 Nome padrão:

```
Projetos
```

📂 2) Organização por área ou curso

Dentro de `Projetos`, você pode separar por contexto:

```
Projetos/
├── Aulas/
├── Pessoais/
├── Estudos/
├── Freelance/
```

📚 Exemplo para seus alunos (ideal)

```
Projetos/
└── Aulas/
    ├── Aula-GitHub/
    ├── Aula-CSharp/
    ├── Aula-BancoDados/
```

📦 3) Estrutura de cada projeto

Cada projeto deve ter sua própria pasta:

```
Aula-GitHub/
│
├── src/
├── docs/
├── README.md
├── .gitignore
```

Etapa 1 – Criar a pasta do projeto no computador

O que fazer:
No Windows, criar uma pasta com o nome do projeto.

Atenção:

- Essa pasta será o projeto do aluno.
- No começo ela é apenas uma pasta comum.
- Ela ainda não é um repositório Git.

Dica de nomenclatura

Explique que o nome da pasta e do repositório deve seguir um padrão profissional:

- letras minúsculas;
- sem acentos;
- sem espaços;
- preferencialmente com hífen.

Exemplos corretos

Plain text

```
aula-github-pratica
cadastro-clientes
sistema-escolar
```

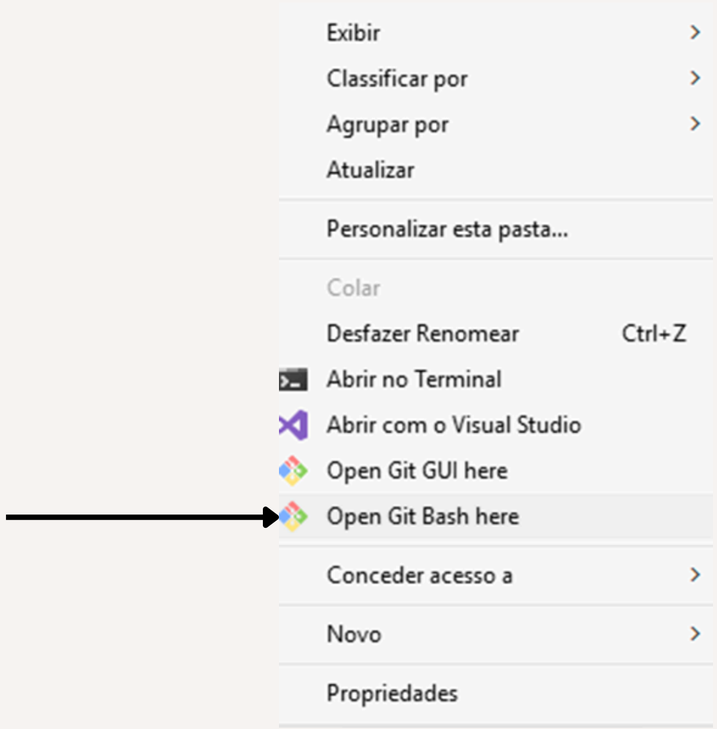
Exemplos incorretos

Plain text

```
Projeto GitHub
Sistema Escolar 2026
MeuProjeto!!!
```

Etapa 2 – Abrir o Git Bash dentro da pasta

O que fazer:
O aluno entra na pasta criada, clica com o botão direito e escolhe:

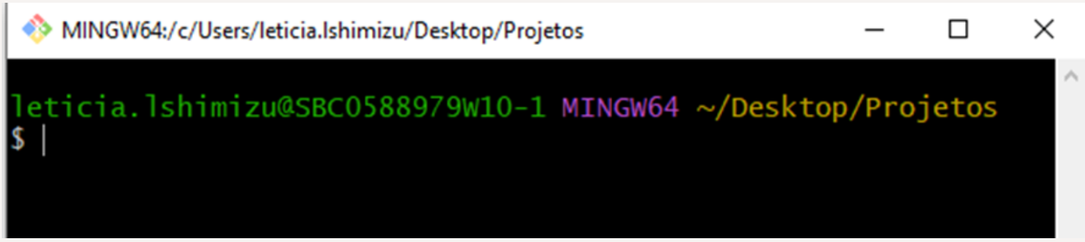


Como explicar

O Git Bash é um terminal que permite usar os comandos do Git.
Podemos dizer:

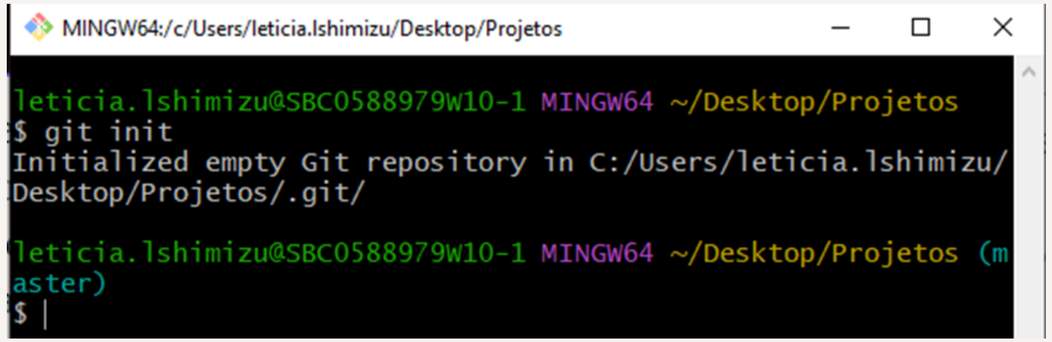
- O Git Bash é a ferramenta que vai permitir conversar com o Git por comandos.

Etapa 3 – Inicializar o repositório Git



Comando:

git init



O que esse comando faz

Esse comando transforma a pasta comum em um repositório Git.

O que acontece internamente

O Git cria uma pasta oculta chamada:

.git

É possível visualizar quando alterar pra exibir arquivos ocultos.

Essa pasta guarda:

- histórico de commits;
- configurações do repositório;
- controle de versões;
- referências de branches.

Como explicar em linguagem simples

A pasta .git é o “cérebro” do versionamento.

É ela que faz o Git começar a acompanhar as mudanças do projeto.

Depois do git init, o terminal normalmente retorna algo parecido com:

<u>Initialized empty Git repository in ...</u>

Isso significa que deu certo.

Etapa 4 – Verificar o estado do repositório

Comando:

git status

```
leticia.lshimizu@SBC0588979W10-1 MINGW64 ~/Desktop/Projetos (m
aster)
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to trac
k)

leticia.lshimizu@SBC0588979W10-1 MINGW64 ~/Desktop/Projetos (m
aster)
$ |
```

O que esse comando faz?

Mostra a situação atual do projeto.

Como explicar

Esse é um dos comandos mais importantes do Git.

Ele mostra:

- em qual branch o aluno está;
- quais arquivos foram alterados;
- quais arquivos ainda não estão sendo monitorados;
- o que está pronto para commit.

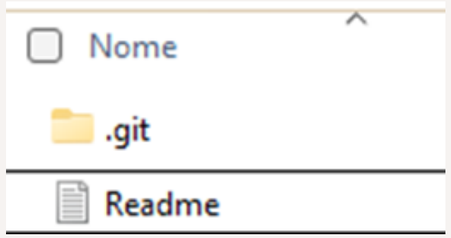
O git status é o comando que o programador consulta toda hora para ver o que está acontecendo no projeto.

Etapa 5 – Criar o primeiro arquivo do projeto

O que fazer

Criar, por exemplo, um arquivo:

Readme.txt



O que é o README

É o arquivo de apresentação do projeto.

Conteúdo inicial simples:

Markdown

```
# Aula GitHub

Projeto criado para prática de Git e GitHub.
```

Concluindo:

O README.md é importante porque:

- apresenta o projeto;
- explica o objetivo;
- ajuda quem acessar o repositório;
- mostra organização.

Podemos dizer:

Todo repositório bom tem pelo menos um README explicando do que se trata.

Etapa 6 – Verificar novamente o status

Comando:

git status

O que o aluno verá:
Agora o Git vai mostrar o arquivo como untracked file.

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
```

O que significa “untracked”?
Significa que o arquivo existe na pasta, mas o Git ainda não está controlando esse arquivo.

Como posso entender?
O Git está dizendo: “Eu vi que existe esse arquivo, mas ainda não fui instruído a acompanhar ele”.

Etapa 7 – Adicionar o arquivo para controle do Git

Comando para arquivo específico:

git add "README.txt"

Ou para todos os arquivos ou vários arquivos:

git add .

O que esse comando faz?
Ele coloca o arquivo na área de preparação, chamada staging area.

O que é"staging area"?
A staging area é como uma área de espera.
O arquivo ainda não virou commit, mas já foi selecionado para entrar no próximo commit.

Diferença importante

- git add README.md → adiciona apenas esse arquivo;
- git add . → adiciona tudo que foi alterado na pasta atual.

Dica didática
No começo, muitos alunos usam git add . porque é mais simples.
Mas é bom explicar que em projetos reais, muitas vezes o ideal é adicionar arquivos específicos.

```
leticia.lshimizu@SBC0588979W10-1 MINGW64 ~/Desktop/Projetos/aulas/aulas-github (master)
$ git add .

leticia.lshimizu@SBC0588979W10-1 MINGW64 ~/Desktop/Projetos/aulas/aulas-github (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   Readme.txt
```

Etapa 8 – Efetuar o primeiro Commit

Comando para arquivo específico:

git commit -m "docs: cria estrutura inicial do projeto"

O que é um commit?

Commit é o registro de uma alteração no projeto.

Ou, de forma mais simples:

Commit é como tirar uma “foto” do projeto naquele momento.

Cada commit salva:

- o estado dos arquivos
- o que foi alterado
- quem fez a alteração
- quando foi feita

Etapa 8 – O que o commit faz na prática?

Quando você executa:

git commit -m "mensagem"

O Git:

1. Salva uma versão do projeto
2. Registra data e hora
3. Identifica o autor
4. Guarda a mensagem descritiva
5. Cria um ponto no histórico

Explicando de forma simples:

Podemos pensar assim:

O commit funciona como um “checkpoint” no projeto.

Sempre que você faz um commit, você cria um ponto de retorno.

Estrutura de um commit

Um commit possui:

- ID único (hash) → código identificador
- Autor
- Data
- Mensagem
- Alterações feitas

Características do commit

É permanente (quase sempre)

Depois que um commit é criado, ele entra no histórico.

Ele não desaparece facilmente.

É incremental

Cada commit representa uma evolução do projeto.

Exemplo:

```
Commit 1 → projeto vazio
Commit 2 → adiciona login
Commit 3 → corrige erro
Commit 4 → adiciona validação
```

Etapa 8 – O que o commit faz na prática?

Características do commit

É rastreável

Você consegue ver:

- quem fez
- o que fez
- quando fez

Permite voltar no tempo

Você pode:

- recuperar versões antigas
- desfazer erros
- comparar mudanças

Depende do git add

Importante ensinar isso:

O commit NÃO pega tudo automaticamente.

Ele só registra o que foi adicionado com:

git add

Erro comum (muito importante explicar)

Aluno acha que commit salva tudo automaticamente.

ERRADO

Fluxo correto:

```
git add .
git commit -m "mensagem"
```

Etapa 9 – Boas práticas com Commit...

1. Commits pequenos

Cada commit deve representar uma mudança específica

ruim:

arrumei tudo

bom:

fix: corrige cálculo do total

2. Mensagens claras

A mensagem deve responder:

O que foi feito?

3. Usar padrão profissional

Formato:

tipo: descrição

Exemplos:

feat: adiciona tela de cadastro

fix: corrige erro no login

docs: atualiza README

refactor: melhora organização do código

Exemplo prático – praticando...

Situação:
Criar um arquivo index.html
Passo 1:
git add index.html

Passo 2:
git commit -m "feat: cria página inicial"

Resultado:

- O arquivo foi registrado no histórico
- Agora existe uma versão salva

Importante!

- Se você não faz commit, você não tem histórico.
- Se você não tem histórico, você não controla seu projeto.
- E se você não controla seu projeto, você não é profissional.

📊 Comparação simples para facilitar entendimento

Conceito	Comparação
Commit	Foto do projeto
Git	Histórico de fotos
Push	Enviar fotos para a internet
Pull	Baixar fotos da internet

Concluindo a 1ª parte:

Resumo:

- Commit = registro de alterações
- Salva o estado do projeto
- Cria histórico
- Permite voltar no tempo
- Precisa do git add antes
- Deve ter mensagem clara

Importante!

- Se você não faz commit, você não tem histórico.
- Se você não tem histórico, você não controla seu projeto.
- E se você não controla seu projeto, você não é profissional.

📊 Comparação simples para facilitar entendimento

Conceito	Comparação
Commit	Foto do projeto
Git	Histórico de fotos
Push	Enviar fotos para a internet
Pull	Baixar fotos da internet